

University Of Dhaka
Computer Science And Engineering

Title:
Multipurpose Calculator Using Digital Logic Circuit

Submitted By

Lab Group : 03
Tanzila Khan (25)
Muhaiminul Islam Ninad (43)
Fayek Ahmed Rahat (57)

Submitted to

Dr. Md. Abdur Razzaque,
Professor, Dept. of CSE, University of Dhaka

Dr. Muhammad Ibrahim,
Assistant Professor, Dept. of CSE, University of Dhaka

Redwan Ahmed Rizvee,
Lecturer, Dept. of CSE, University of Dhaka

Contents

Contents ii

List of Figures ii

List of Tables iii

1 Introduction 1
- 1.1 Description . . . 1
- 1.2 The Rationale . . . 2
- 1.3 Objectives . . . 2
- 1.4 Functionality . . . 3

2 Components 4
- 2.1 74LS283 4-bit Adder: . . . 4
- 2.2 74LS86 Quad 2-input Exclusive-OR Gate: . . . 4
- 2.3 74LS08 Quad 2-Input AND Gate: . . . 5
- 2.4 74LS32 Quad 2-Input OR Gate: . . . 5
- 2.5 Essential hardware components: . . . 5

3 Implementation 6
- 3.1 7-Bit Parallel Adder: . . . 6
- 3.2 6-Bit Parallel Subtractor: . . . 8
- 3.3 4 by 2-Bit Parallel Multiplier: . . . 12
- 3.4 Demonstration Video . . . 14

4 Challenges and Conclusion 15
- 4.1 Challenges . . . 15
- 4.2 Conclusion . . . 16

List of Figures

2.0.1 7408, 7486, 7432 and 74283 IC Pinout 4

3.1.1 Circuit Diagram of Full Adder 7
3.1.2 Circuit Diagram of 7-bit Parallel Adder using Full Adders 8
3.2.1 Circuit Diagram of Full subtractor 9
3.2.2 Circuit Diagram of 7-bit Parallel Adder using Full Adders 11
3.3.1 Circuit Diagram of Multiplier Circuit 13

List of Tables

3.1.1 Full Adder 6
3.2.1 Full Subtractor 8
3.2.2 XOR Operation 11
3.3.1 AND Operation 12

Chapter 1

Introduction

In the realm of digital electronics, the pursuit of efficient computational devices has been relentless. This project amalgamates theoretical knowledge and hands-on application to birth a multipurpose calculator, emphasizing the significance of digital logic circuits in arithmetic operations.

1.1 Description

The project aimed to create a versatile calculator employing digital logic circuits to perform arithmetic operations such as addition, subtraction, and multiplication on binary numbers. This calculator integrates a 7-bit parallel adder, 6-bit parallel subtractor, and a 4-bit by 2-bit parallel multiplier. Here addition operation occurs between two 7-bit numbers, subtraction operation occurs between two 6-bit numbers and multiplication operation occurs between a 4 bit and a 2 bit numbers.

A parallel adder is a type of digital circuit that can calculate the arithmetic addition of two binary integers longer than one bit. It is made up of a series of full adders connected in a chain, with each full adder's output carry being connected to the carry input of the full adder that comes after it in the chain. To execute this operation, a n-bit parallel adder needs n complete adders.

A parallel subtractor is a digital circuit designed to efficiently compute the arithmetic difference between two binary integers, particularly when the length of the integers exceeds one bit. This circuit performs the subtraction operation simultaneously across all corresponding bits of the two input integers, leveraging parallel processing to enhance computational speed and accuracy. The result is a precise determination of the difference between the two binary numbers, making

parallel subtractors valuable components in digital arithmetic circuits for handling multi-bit binary subtraction.

In the parallel multiplier, the process begins by generating partial products through the multiplication of each bit of the multiplicand with the corresponding bit of the multiplier. These partial products are calculated simultaneously, leveraging parallel processing for efficiency. Subsequently, the resultant product, denoted as P, is obtained by adding together these partial products in parallel. This parallelized approach accelerates the multiplication process, optimizing computational speed and efficiency.

1.2 The Rationale

Arithmetic operations serve as the foundation of computation, and this project aims to streamline these operations in the binary domain. By leveraging fundamental principles of digital electronics, the objective is to create a calculator capable of executing addition, subtraction, and multiplication operations on binary numbers.

1.3 Objectives

- **Construction of a 7-bit adder :** Utilizing 74LS283 ICs to perform addition operations accurately on 8-bit binary numbers.
- **Design of a 6-bit subtractor:** Employing the 2's complement method and the 8-bit adder for subtraction operations.
- **Creation of a 4 by 2-bit multiplier:** Utilizing combinational logic circuits to perform bit-wise multiplication of 4*2-bit binary numbers.
- **Verification through Testing:** Validating the functionalities of the calculator by executing diverse arithmetic operations on various input combinations.

1.4 Functionality

- **7-Bit Parallel Adder Functionality:** It performs arithmetic addition on two 7-bit binary numbers. It utilizes a series of full adders connected in parallel. It accommodates simultaneous processing of all corresponding bits for efficient addition.

- **6-Bit Parallel Subtractor Functionality:** It computes the arithmetic difference between two 6-bit binary numbers. It is implemented using the 2's complement method for subtraction. It utilizes XOR gates to toggle bits in the subtrahend, enabling seamless integration into the overall calculator.

- **4 by 2-Bit Parallel Multiplier Functionality:** It executes multiplication operation on a 4-bit multiplicand and a 2-bit multiplier. It generates partial products through parallel processing of individual bits. It achieves efficiency by adding partial products in parallel to obtain the resultant product.

- **Combinational Logic Integration:** It uses combinational logic to merge diverse functionalities and provides a unified approach for addition, subtraction, and multiplication. It also enhances adaptability for different computational requirements.

- **Versatility and Adaptability:** Our project accommodates various arithmetic operations on binary inputs. It serves as a multipurpose calculator for different binary calculations. It demonstrates versatility in digital electronics applications.

- **Accurate and Efficient Operations:** It ensures precision in binary arithmetic operations. This project leverages parallel processing for computational speed. It maintains efficiency in addition, subtraction, and multiplication.

- **Seamless Integration:** This project integrates components cohesively for smooth communication. It ensures compatibility between different arithmetic modules.

Chapter 2

Components

Below are the list of components used to actualize this project.

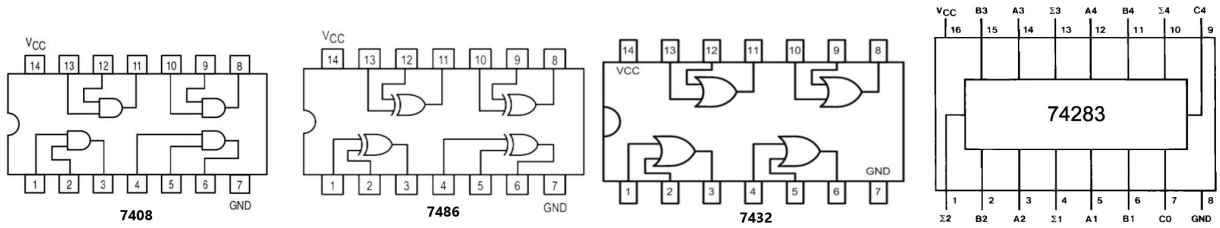


Figure 2.0.1 – 7408, 7486, 7432 and 74283 IC Pinout

2.1 74LS283 4-bit Adder:

74LS283(Figure 2.0.1) is a high-speed 4-Bit Binary Full Adder with internal carry lookahead. It accepts two 4-bit binary words (A1–A4, B1–B4) and a Carry Input (C0). It generates the binary Sum outputs (S1–S4) and the Carry Output (C4) from the most significant bit.

2.2 74LS86 Quad 2-input Exclusive-OR Gate:

This IC(Figure 2.0.1) is use to perform EXCLUSIVE-OR gate logic function in circuit. Input/output pins are placed in pair of 3 pins(2 pin for input and 1 for result/output), one IC can connect up-to 4 devices.

2.3 74LS08 Quad 2-Input AND Gate:

IC 7408(Figure 2.0.1) is a logic gate chip with four two-input AND gates. Each of the four AND gates are independent. Each gate has one output, two inputs and three pins.

2.4 74LS32 Quad 2-Input OR Gate:

The 7432 IC (Figure 2.0.1) provides four independent 2-input OR gates with standard push-pull outputs. Inputs include clamp diodes.

2.5 Essential hardware components:

Breadboard, wires, power supply (5V DC), LED bulbs, switch and resistors(1 k-Ohm)

Chapter 3

Implementation

3.1 7-Bit Parallel Adder:

We implemented a 7 bit parallel adder that could handle the addition operation of two 7 bit numbers. Below is the detailed explanation of the process.

C_i	A	B	Sum	C_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 3.1.1 – Full Adder

First we started with making a Full Adder Circuit. In Table 3.1.1, A and B are the input bits, C_i is the initial carry bit, C_o is the carry from adding A and B. Using SOP, we get the following expressions(3.1 and 3.1) for the sum and carry-out,

$$\begin{aligned} Sum &= A^{'}BC_i^{'} + AB^{'}C_i^{'} + A^{'}B^{'}C_i + ABC_i \\ &= C_i^{'}(A^{'}B + AB^{'}) + C_i(A^{'}B^{'} + AB) \\ &= C_i^{'}(A \oplus B) + C_i(A \oplus B)^{'} \\ &= A \oplus B \oplus C_i \end{aligned} \tag{3.1}$$

$$\begin{aligned} C_o &= ABC_i^{'} + AB^{'}C_i + A^{'}BC_i + ABC_i \\ &= C_i(A^{'}B + AB^{'}) + AB(C_i^{'} + C_i) \\ &= C_i(A \oplus B) + AB \end{aligned} \tag{3.2}$$

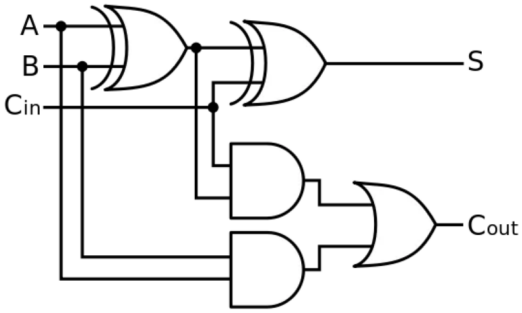


Figure 3.1.1 – Circuit Diagram of Full Adder

We then created a full adder using the above expressions. But as this has its scalability issues, we strategically utilized two 4-bit adder IC-74283(Figure 2.0.1) components to construct the circuit of 7-bit parallel adder.

The design involves the integration of seven switches dedicated to the augend number (A), a switch for initial carry input and an additional seven switches for the addend number (B). Ensuring the proper functioning of the integrated circuits, we provided them with VCC and GND connections.

The interconnection of the sum outputs from each full adder was executed in a systematic sequence, creating a seamless flow. Visualizing the output, we employed LEDs to represent the result, with each LED corresponding to a specific bit in the sum. This meticulous arrangement not only ensures clarity in the output representation but also enhances the visual appeal of the circuit.

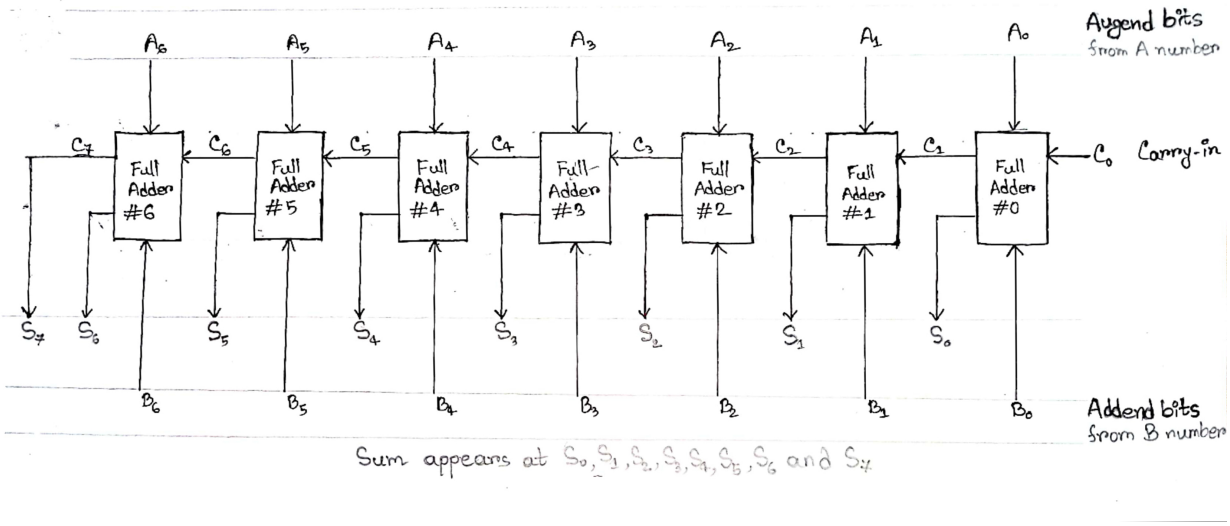


Figure 3.1.2 – Circuit Diagram of 7-bit Parallel Adder using Full Adders

3.2 6-Bit Parallel Subtractor:

Leveraging 2's complement addition method, we then integrated the 7-bit adder to perform subtraction operations. The procedure is as follows.

We started with

Borrow-in(B_i)	A(input)	B(input)	Diff	Borrow out(B_o)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	0
1	0	0	1	1
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

Table 3.2.1 – Full Subtractor

$$\begin{aligned} D &= A'BBi' + AB'Bi' + A'B'Bi + ABBi \\ &= Bi(A'B + AB') + Bi(A'B' + AB) \\ &= Bi'(A \oplus B) + Bi(A \oplus B)' \\ &= A \oplus B \oplus Bi \end{aligned} \tag{3.3}$$

$$\begin{aligned} C_o &= A^{'}BBi^{'} + A^{'}B^{'}Bi + A^{'}BBi + ABBi \\ &= Bi(A^{'}B^{'} + AB) + A^{'}B(Bi^{'} + Bi) \\ &= Bi(A \oplus B)^{'} + A^{'}B \end{aligned} \tag{3.4}$$

Here A and B are the input bits, Bi is the initial borrow bit, Bo is the borrow from subtracting B from A. Using SOP, we get the expressions for the difference and borrow-out,

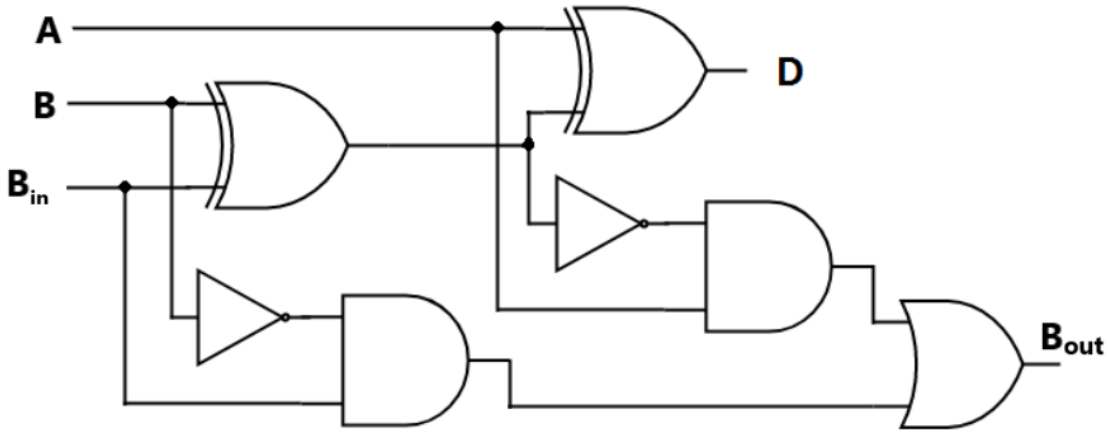


Figure 3.2.1 – Circuit Diagram of Full subtractor

An interesting point is that the sum(or difference) part of binary addition (or subtraction) gives the same result using XOR operation. Only the carry(or borrow) is different. With this property, we can add and subtract binary numbers if we modify our circuit. In fact, binary subtraction is just repeated XOR operation while keeping track of the carry-overs.

Binary subtraction is just the binary addition of a negative number. In binary subtraction, we are taking the 2's complement of the subtrahend number and adding it to the minuend number, from which we want to subtract the subtrahend.

For example: $(A-B) = A + (2's \text{ complement of } B)$

In our project, we developed a 6-bit subtractor utilizing the 2's complement method for performing subtraction between two 6-bit numbers. To incorporate the sign information, we introduced an additional bit as the 7th bit, serving as the sign bit. When the sign bit is 0, it indicates a

positive number, and when it is 1, it signifies a negative number.

The 1's complement of a binary number is derived by inverting all its bits, and the 2's complement is obtained by adding 1 to the 1's complement.

Let's look at the following operation: $5512 = 43$

The numbers in the above operation can be denoted in binary as follows:

$$\begin{aligned}(55)_{10} &\rightarrow (110111)_2 \\ (12)_{10} &\rightarrow (001100)_2\end{aligned} \tag{3.5}$$

On subtraction, the difference $(43)_{10}$ would be represented as: $(110111)_2 - (001100)_2 = (101011)2$

Now, we would do this operation using 2's complement method. The given numbers with its sign bits are,

$$\begin{aligned}(55)_{10} &\rightarrow (0110111)_2 \\ (12)_{10} &\rightarrow (0001100)_2\end{aligned} \tag{3.6}$$

We get the 1's complement of the subtrahend number by toggling the bits,

$(12)_{10} \rightarrow (1110011)_2$

So, The 2's complement will be, $(1110011)_2 + (1)_2 = (1110100)_2 = (-12)_{10}$

Now adding the subtrahend number to the minuend number, we get,

$$\begin{aligned}(55)_{10} &\rightarrow (0110111)_2 \\ +(-12)_{10} &\rightarrow (1110100)_2 \\ \hline (43)_{10} &\rightarrow (10101011)_2\end{aligned} \tag{3.7}$$

Here, the 8th bit of the answer is the overflowed carry.It is discarded. So, the sign bit(7th bit) of the answer is 0. It means the answer is a positive number.

In the development of our 7-bit parallel adder-subtractor, a key aspect involves the computation of the 2's complement of the subtrahend during subtraction. To achieve this, we strategically employed XOR gates to toggle the bits of the subtrahend (B). The use of XOR gates provides a controlled mechanism for bit toggling, allowing for precise manipulation.

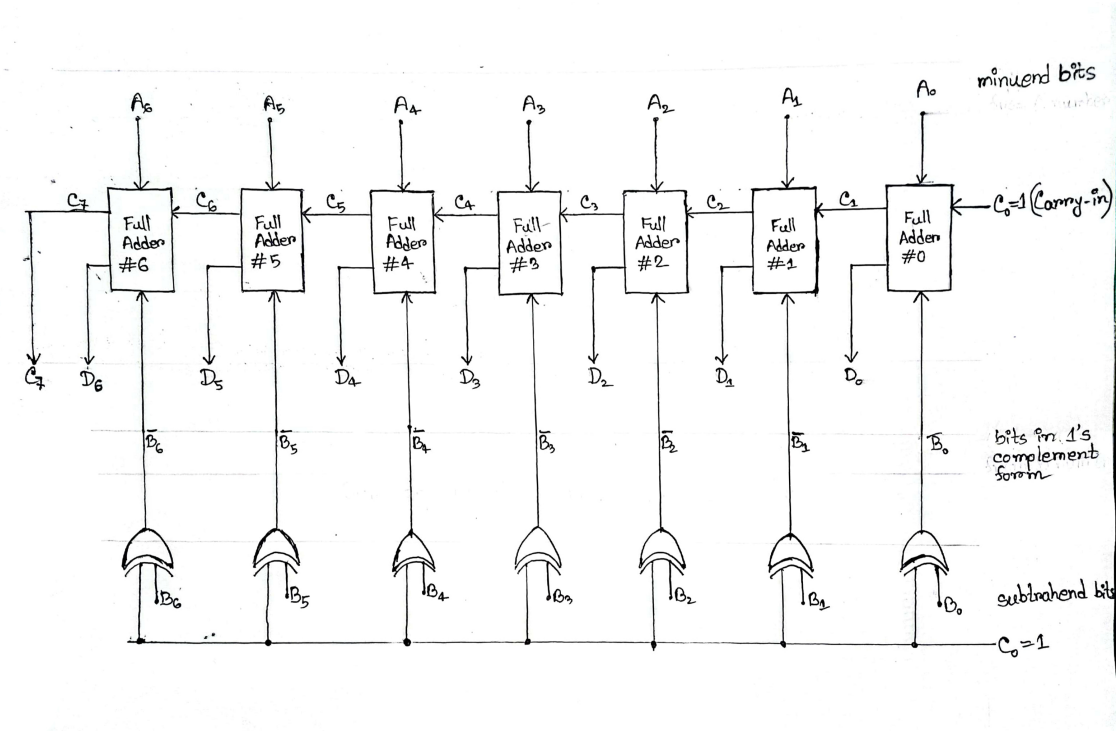


Figure 3.2.2 – Circuit Diagram of 7-bit Parallel Adder using Full Adders

In this configuration, each XOR gate takes as inputs the initial Carry-in (Ci) of the addition operation and a corresponding bit of the subtrahend (B). The outputs of these XOR gates serve as the modified input bits for the addend B in our previously implemented 7-bit parallel adder.

C_i	B_1	Output	State
0	0	0	Unchanged
0	1	1	Unchanged
1	0	1	Toggled
1	1	0	Toggled

Table 3.2.2 – XOR Operation

This arrangement allows us to control the toggling of the bits of number B based on the addition or subtraction operation being performed. If we keep our initial carry-in(C_i) 0, then the bits of the number B will not toggle. In this condition, the circuit can perform addition operation only. In case of initial carry-in(C_i) becoming 1, the bits of the number B will toggle because of the XOR gates. This makes the 2's complement of the number B. In this condition, this same circuit will actually perform only subtraction operation using 2's complement method.

3.3 4 by 2-Bit Parallel Multiplier:

We also implemented the ability to multiply two numbers in our calculator as multiplication is a crucial part of any calculator. The process we implemented to multiply two numbers is similar to the one taught in high school.

$$\begin{array}{rcccc} & & & 4 & 3 \\ \times & & & 2 & 3 \\ \hline & & 1 & 2 & 9 \\ & & 8 & 6 & 0 \\ \hline & & 9 & 8 & 9 \end{array}$$

In this process, first we multiply the multiplicand with the rightmost digit of the multiplier and write it down. Then multiply the multiplicand with the 2nd rightmost digit of the multiplier and write the product (partial product) with 1 digit shifted to the left. And this process goes on for all the digits of the multiplier. After we are done with this step we add all the products to get the final product.

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Table 3.3.1 – AND Operation

In the case of binary number, the process remains the same. But, the bitwise multiplication can be replaced by logical AND (Table: 3.3.1) as they have the same truth table.

To multiply two binary numbers i.e. 1101 and 110 we write them down like conventional multiplication

			1	1	0	1
×				1	1	0
			0	0	0	0
		1	1	0	1	x
	1	1	0	1	x	x
1	0	0	1	1	1	0

To accomplish this with digital circuits, we take the input of the two numbers and pairwise AND the bits of both the numbers. This requires at most n×m logic gates where n and m is the number of bits in the multiplicand and the multiplier. Then we add the partial products using the same principle as Adder that we also implemented separately.

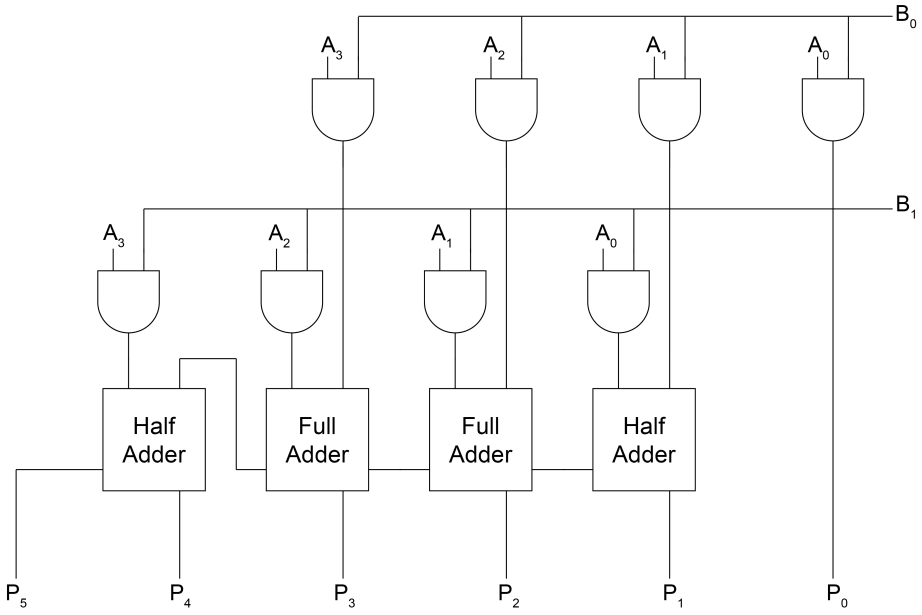


Figure 3.3.1 – Circuit Diagram of Multiplier Circuit

As the number of logic gates required increases quadratically with the bits of the numbers, we implemented a 4 bit by 2 bit multiplier That can multiply one number with 4 bits and another number with 2 bits. The circuit diagram of the multiplier is as figure 3.3.1.

In the figure, the bits of the 4-bit number is represented as A0 to A3 and the 2-bit number is represented with B0 and B1 At first we ANDed the bits with AND gate IC 7408, which has pinout as figure 2.0.1.

7408 is a Quad 2 Input AND Gate We needed 2 such ICs to get the AND part done.

Then we hooked the outputs of this IC to the Full Adder IC 74283 which has a pinout as figure 2.0.1.

As 74283 can add two 4-bit numbers, we needed only one of this IC and wired everything as the circuit diagram. As 74283 IC is a full adder IC, so we converted the 4th full adder into a half adder using 0 as a input of that full adder.

And that completed our Multiplier circuit.

3.4 Demonstration Video

Here we attached a google drive link. This drive contains videos that shows our implemented circuit works smoothly as a binary calculator.

Google Drive Link to the Video Folder

Chapter 4

Challenges and Conclusion

4.1 Challenges

- **Bit-Level Precision:** Managing the precision of bits in each operation, especially during subtraction where 2's complement is involved, was challenging to avoid errors or unexpected results.
- **Power and Ground Integrity:** Providing stable power and ground connections to the integrated circuits is vital. Inadequate power supply or ground connections resulted in unreliable behavior of the digital circuits.
- **Testing and Debugging:** Identifying and rectifying errors in such a multifaceted system requires systematic testing procedures. Given the complexity of the project, testing and debugging the entire system was very time-consuming.
- **Scaling for Larger Numbers:** We faced difficulties handling two seven-bit binary numbers. Extending the calculator's capability to handle larger binary numbers might pose challenges in terms of circuit complexity, timing considerations, and resource utilization.

4.2 Conclusion

The conclusion encapsulates the successful integration and functionality of the designed circuits, demonstrating precise arithmetic operations encompassing addition, subtraction, and multiplication on binary numbers. It highlights the effectiveness and accuracy of digital logic circuits in performing intricate computational tasks.